

# **Research Topic : Design and Evaluation of an Adversarial Multi-Agent Framework for Automated Detection, Remediation and Verification of REST API Vulnerabilities**

## **CHAPTER 1**

### **INTRODUCTION**

#### **1.1 Background of the Study**

Modern web applications are increasingly built as collections of REST APIs rather than as single monolithic servers. This shift has moved a large part of the attack surface away from the browser and towards the API layer itself. Frameworks such as FastAPI have made it considerably easier to expose an endpoint, but the security responsibilities that come with that endpoint — verifying who is asking, what object they are allowed to touch, and which fields they may write — still rest entirely with the developer.

The OWASP API Security Top 10 (2023 edition) reflects this reality. Its highest-ranked risk, Broken Object Level Authorization, is not a syntactic defect that a compiler or linter can catch. A vulnerable endpoint is usually well-formed code that authenticates the caller correctly and then simply forgets to check ownership of the requested object. The resulting request is syntactically valid, passes a web application firewall, and returns HTTP 200. Only the semantics are wrong.

Traditional automated scanners struggle with this category of flaw for precisely that reason. Tools such as OWASP ZAP are effective at detecting pattern-based issues, but authorization logic is specific to the application's own data model and cannot be matched by a generic signature. At the same time, large language models have begun to show genuine capability in reading and reasoning about source code, which raises an obvious question: can an LLM-driven system find and repair the class of vulnerabilities that signature-based scanners systematically miss?

Early attempts to answer that question have generally taken the form of a single model prompted to review code and report problems. This project takes a different approach. It investigates whether separating the work across several specialised agents — one that understands the target, one that attacks it, one that independently judges the evidence, and one that repairs the code — produces more trustworthy results than a single general-purpose agent working alone. The platform developed for this study, named ClairSec, implements that pipeline and executes every test against the real application running inside an isolated container, so that a reported vulnerability is backed by an actual exploited request rather than by a model's opinion about the source.

#### **1.2 Problem Statement**

Automated security tooling for REST APIs currently sits between two unsatisfactory options. Signature-based scanners are deterministic and reproducible, but they cannot reason about application-specific authorization logic and therefore miss the most prevalent class of API vulnerability. LLM-based tools can

reason about that logic, but they are prone to reporting issues that do not actually exist, because nothing in the process forces the model's claim to be tested against the running application.

This second failure mode is more damaging than it first appears. A security tool that produces a high proportion of false positives consumes reviewer time, and once practitioners learn to distrust its output they stop acting on it altogether. The problem is compounded when the same model that proposes a finding is also asked to confirm it, since the model has no independent basis on which to disagree with itself.

A parallel weakness exists on the remediation side. Where automated repair is attempted at all, success is typically declared when the original exploit stops working. That criterion is trivially satisfiable by a patch that disables or deletes the affected endpoint, which blocks the exploit perfectly while destroying the functionality the endpoint existed to provide. Reported repair rates that rely on this single criterion therefore overstate real performance.

Finally, feeding untrusted third-party source code into a model that subsequently writes patches to disk introduces a security concern in the tool itself. Instructions embedded in a target repository's comments or documentation may attempt to influence the agent's behaviour, a risk that existing work in this area rarely addresses. This project responds to these three gaps by requiring runtime evidence for every finding, by adjudicating that evidence with an agent structurally separated from the one that produced it, and by treating a repair as successful only when it both blocks the exploit and leaves the application's legitimate behaviour intact.

### **1.3 Objectives of the Study**

The overall aim of this research is to design, implement and evaluate a multi-agent platform that detects, repairs and verifies REST API vulnerabilities without requiring the user to trust an unverified model assertion. The specific objectives are:

- **Agent Architecture:** To design and implement a four-stage pipeline consisting of a Builder, an Attacker, an Evaluator and a Fixer, in which each agent has a single clearly bounded responsibility and communicates through typed records rather than free-form text.
- **Runtime Evidence:** To ensure that no vulnerability is reported as confirmed unless a real request against the running target produced an observable violation, with the decision made by deterministic platform code rather than by a language model.
- **Independent Evaluation:** To structurally separate the agent that proposes a finding from the agent that adjudicates it, so that the Evaluator receives only the captured request, response and oracle outcome, and never the Attacker's own narrative.
- **Automated Remediation:** To generate reviewable source patches for confirmed findings and apply them exclusively within an isolated workspace, leaving the user's original project byte-for-byte unmodified.
- **Dual-Criterion Verification:** To re-test each applied patch against both the original exploit and the application's functional test suite, so that a repair which breaks legitimate behaviour is recorded as a regression rather than as a success.

- **Controlled Execution:** To execute all target code inside hardened, network-isolated containers, ensuring that analysing an untrusted project cannot compromise the host.

## **1.4 Scope of the Study**

This study is limited to REST APIs written using the FastAPI framework in Python. Restricting the target framework allows the Builder agent to use reliable static analysis of route decorators and Pydantic models rather than attempting to infer structure heuristically across many frameworks.

Three vulnerability classes are addressed in the implemented system: Broken Object Level Authorization, mass assignment through unvalidated request fields, and security misconfiguration including debug exposure and permissive cross-origin policies. These were selected because each can be confirmed unambiguously at runtime by observing the response to a controlled request. Classes that require multi-service infrastructure or business-specific ground truth, such as server-side request forgery and abuse of sensitive business flows, are documented as out of scope.

All execution takes place locally. Target applications run in Docker containers on the developer's own machine, and no request is ever issued against third-party or production infrastructure. The platform is intended for projects the user is authorised to test, and the desktop client requires that confirmation at import.

The work covers the construction of the platform and its evaluation against a controlled benchmark application containing deliberately planted vulnerabilities. Large-scale comparative experiments against traditional scanners and a wider corpus of real-world repositories are defined in the research design but fall outside the implementation reported in these chapters.

## **1.5 Significance of the Study**

The practical value of this work lies in producing security findings that a developer can act on immediately. Because every confirmed finding carries the exact request that triggered it and the response that proved it, a reviewer can reproduce the issue without re-deriving it from a textual description. This addresses the credibility problem that limits the usefulness of many automated tools.

The architectural contribution is the deliberate separation between proposing a finding and judging it. Most current systems collapse these roles into a single model. By giving the Evaluator access only to captured evidence, and by computing the pass or fail decision in ordinary program code rather than in a prompt, the design removes the possibility of a model confirming its own unsupported claim.

The dual-criterion verification requirement is significant for the automated repair literature more broadly. Reporting the proportion of patches that block an exploit without simultaneously reporting how many of those patches broke working functionality produces figures that cannot be compared meaningfully between systems. This project treats functional regression as a first-class outcome and records it separately.

Finally, because the platform necessarily supplies untrusted third-party source code to a language model that then acts on the results, it provides a natural setting in which to examine whether role separation

affects susceptibility to instructions planted in the code under test. This question has received comparatively little attention and is identified here as a secondary line of enquiry.

## **1.6 Research Questions**

The study is guided by the following questions:

- RQ1 (Detection): Can a multi-agent pipeline that requires runtime evidence correctly identify planted authorization, mass assignment and misconfiguration vulnerabilities in a controlled FastAPI benchmark application?
- RQ2 (False Positives): Does requiring an independent Evaluator and a deterministic oracle prevent the system from reporting vulnerabilities against a functionally equivalent application in which the same endpoints have been correctly secured?
- RQ3 (Repair Quality): What proportion of automatically generated patches both block the original exploit and leave the application's functional test suite passing, and how many satisfy only the first of these conditions?
- RQ4 (Robustness of Repair): Do verified patches also withstand an adapted variant of the original attack, indicating that the underlying weakness was addressed rather than the specific exploit path?
- RQ5 (Architectural Contribution): Which properties of the observed behaviour can be attributed specifically to the separation of agent roles, as distinguished from the capability of the underlying language model?

## **1.7 Organization of the Report**

The remainder of this report is arranged as follows:

- Chapter 1 (Introduction) establishes the background of API security, states the problem, and defines the objectives, scope and research questions.
- Chapter 2 (Literature Review) surveys existing work on automated vulnerability detection, LLM-based repair agents, benchmark design and prompt-injection defence, and identifies the gaps this project addresses.
- Chapter 3 (Methodology) describes the research design, the data structures and benchmark applications used, the tools selected, the responsibilities assigned to each agent, and the overall system architecture.
- Chapter 4 (Implementation and Results) will present the constructed platform and the measurements obtained from the benchmark runs.
- Chapter 5 (Discussion) will interpret those measurements with reference to the research questions and consider the limitations of the approach.
- Chapter 6 (Conclusion and Future Work) will summarise the contribution and outline the extensions required for larger-scale evaluation.

## CHAPTER 2

### LITERATURE REVIEW

#### 2.1 Overview of the Domain

Automated detection of software vulnerabilities has a long history, but the application of large language model agents to the problem is recent and has developed rapidly. The field currently divides into three loosely connected strands: classical dynamic scanning, LLM-based detection and reasoning, and automated program repair.

Classical dynamic application security testing tools, of which OWASP ZAP is the most widely deployed open-source example, operate by sending crafted requests and matching responses against known signatures. Their principal strengths are determinism and reproducibility. Their principal limitation, widely acknowledged in practice, is an inability to reason about authorization logic that is specific to an individual application's data model.

The second strand applies language models to source code. Work in this area has moved from simple classification of vulnerable functions towards agentic systems that can navigate a repository and use analysis tools. Results have been mixed. Evaluations built on real Common Vulnerabilities and Exposures records report that even capable software engineering agents repair only a minority of cases, and an influential earlier study argued that language models cannot yet reliably reason about security properties in isolation.

The third strand concerns automated program repair. Here the methodological trend is clearly towards execution-based validation: a patch is assessed by running a proof of vulnerability and a regression suite, rather than by measuring textual similarity to a reference fix. This project adopts that position and extends it by treating the functional-regression outcome as a separately reported metric rather than as an implicit precondition.

A fourth concern cuts across all three strands. Once an agent both reads untrusted input and performs consequential actions, instructions embedded in that input become an attack vector. Recent work on securing LLM agents proposes architectural patterns that constrain what an agent is able to express, in preference to filtering what it reads. The platform described in this report adopts that principle directly.

#### 2.2 Review of Related Work

##### 2.2.1 Multi-Agent Architectures for Vulnerability Detection

Several recent systems decompose vulnerability analysis across cooperating agents rather than issuing a single prompt. Hypothesis-and-validation designs generate candidate explanations for suspicious code and then attempt to confirm them, while adversarial designs assign opposing roles in order to surface weak reasoning. The reported motivation in this literature is consistently the reduction of false positives. What is generally absent, however, is a structural guarantee that the validating component is genuinely

independent of the proposing one; in most designs both roles are performed by the same model over a shared conversation, so the validator inherits the proposer's framing.

### **2.2.2 Benchmarks and Ground Truth**

Evaluation of these systems depends on the benchmark used. Recent benchmarks constructed from real CVE records provide realism and executable test environments, a clear improvement on datasets consisting only of isolated functions. They carry a recognised weakness, however: because the corresponding fix commits are public, they are likely to appear in model pretraining data, and a model may therefore recall a fix rather than derive it. This contamination risk has prompted a move towards held-out or synthetically reintroduced vulnerabilities. The present study responds by constructing its benchmark application specifically for this research, with each vulnerability planted and recorded before any scan is executed.

### **2.2.3 Automated Repair and Its Evaluation**

Automated repair systems for security defects are commonly evaluated by whether a proof-of-vulnerability exploit ceases to succeed after patching. The stronger studies pair this with a regression suite and require both conditions to hold. This dual criterion matters because the two conditions can be satisfied in opposition: the simplest way to defeat an exploit is to remove the functionality it targets. Where only the exploit condition is reported, published repair rates are not directly comparable. This project adopts the dual criterion and additionally records a variant-attack outcome, in order to distinguish a patch that closes one exploit path from one that addresses the underlying weakness.

### **2.2.4 Prompt Injection and Agent Containment**

Where an agent processes untrusted content and can act on the result, prompt injection becomes an execution concern rather than merely a content concern. The current consensus in this literature is that detection-based filtering is inherently probabilistic and cannot be relied upon alone, and that sandboxing together with architectural constraint of the agent's available actions provides the more dependable control. Proposed design patterns include quarantining untrusted input in a restricted extraction step whose output is a strictly typed structure, and limiting the agent to a predetermined set of permitted operations. Both patterns are implemented in the platform described here.

## **2.3 Existing Methodologies and Tools**

The dominant methodology in LLM-based security research follows a broadly linear sequence: supply source code or a running target to a model, obtain a set of reported findings, and compare those findings against a labelled ground truth. Variations concern the prompting strategy, the tools made available to the agent, and the granularity of the labels.

This project deliberately departs from that pattern in one respect. Rather than treating the model's output as the finding, the model's output is treated as a proposal that must be substantiated by executing a request against the running application. The decision as to whether a violation occurred is made by ordinary program code inspecting the recorded request and response. This design choice is

what allows detection performance to be attributed to the system rather than to the model's self-consistency.

The implementation uses Python with FastAPI for the orchestration backend, Docker for target isolation, MongoDB for structured scan records, and Flutter for the desktop client. The language model is accessed through a provider abstraction so that the specific model can be substituted without altering agent logic, and every call is recorded with its model identifier, prompt version and token usage to support reproducibility.

## 2.4 Gap Analysis

| Sr. No. | Reference | Paper Title                                                                                                     | Year | Methodology                                                                                                                                                      | Key Findings                                                                                                                           | Research Gap                                                                                                                                                       |
|---------|-----------|-----------------------------------------------------------------------------------------------------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1       | [1]       | CVE-Bench: Benchmarking LLM-based Software Engineering Agent's Ability to Repair Real-World CVE Vulnerabilities | 2025 | Constructed an executable benchmark of real CVEs across multiple languages and repositories, providing agents with a test environment and static analysis tools. | Even capable software engineering agents repaired only a minority of vulnerabilities; agents used available analysis tools poorly.     | Benchmark is built from public CVEs whose fix commits may appear in pretraining data, and the study does not separate exploit blocking from functional regression. |
| 2       | [2]       | VulAgent: Hypothesis-Validation based Multi-Agent Vulnerability Detection                                       | 2025 | Decomposed detection into hypothesis generation and validation stages performed by cooperating agents.                                                           | Role decomposition reduced false positives relative to single-pass prompting.                                                          | Validation is performed by the same model within a shared context, so independence is nominal; findings are not confirmed by executing the application.            |
| 3       | [3]       | Design Patterns for Securing LLM Agents against Prompt Injections                                               | 2025 | Proposed architectural patterns including quarantined processing of untrusted input and restriction of agent action space.                                       | Constraining what an agent can express is more dependable than filtering what it reads; detection-based defences remain probabilistic. | Patterns are presented in general form and are not evaluated in a security-testing tool that necessarily ingests untrusted third-party source code.                |
| 4       | [4]       | OWASP API Security Top 10 (2023 Edition)                                                                        | 2023 | Ranked API security risks by prevalence, exploitability and impact based on industry data.                                                                       | Broken Object Level Authorization remains the highest-ranked API risk and appears in a large share of observed                         | Defines risk categories but provides no automated detection method; authorization flaws remain application-specific and resist                                     |

|   |     |                                                                     |      |                                                                                                                  |                                                                                                                                        |                                                                                                                                                                |
|---|-----|---------------------------------------------------------------------|------|------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   |     |                                                                     |      |                                                                                                                  | attacks.                                                                                                                               | signature matching.                                                                                                                                            |
| 5 | [5] | Do Language Models Reason About Security Vulnerabilities?           | 2024 | Evaluated language models on security reasoning tasks using a structured benchmark and controlled perturbations. | Models were frequently inconsistent and sensitive to superficial changes, indicating limited reliable security reasoning in isolation. | Establishes that model assertion alone is insufficient, but does not propose an architecture in which claims are substantiated by execution.                   |
| 6 | [6] | OWASP ZAP and Schemathesis (Traditional and Schema-Driven Scanning) | 2024 | Signature-based dynamic scanning and OpenAPI schema-driven property testing of live endpoints.                   | Effective and reproducible for pattern-detectable defects such as injection and misconfiguration.                                      | Cannot reason about per-application ownership semantics, so authorization flaws that return valid, authenticated HTTP 200 responses are systematically missed. |

The review therefore identifies the following gaps, which this project addresses:

1. Reported findings are frequently based on model assertion rather than on demonstrated runtime behaviour, leaving no reproducible artefact for a reviewer to inspect.
2. Where a validation stage exists, it is rarely structurally independent of the component that produced the finding, so it cannot meaningfully disagree with it.
3. Repair success is commonly measured by exploit blocking alone, which cannot distinguish a correct fix from one that removes the affected functionality.
4. Benchmarks derived from public vulnerability records are exposed to pretraining contamination, weakening claims about genuine detection capability.
5. Few systems that supply untrusted source code to an acting model address the possibility that instructions embedded in that source may influence the agent's behaviour.



## CHAPTER 3

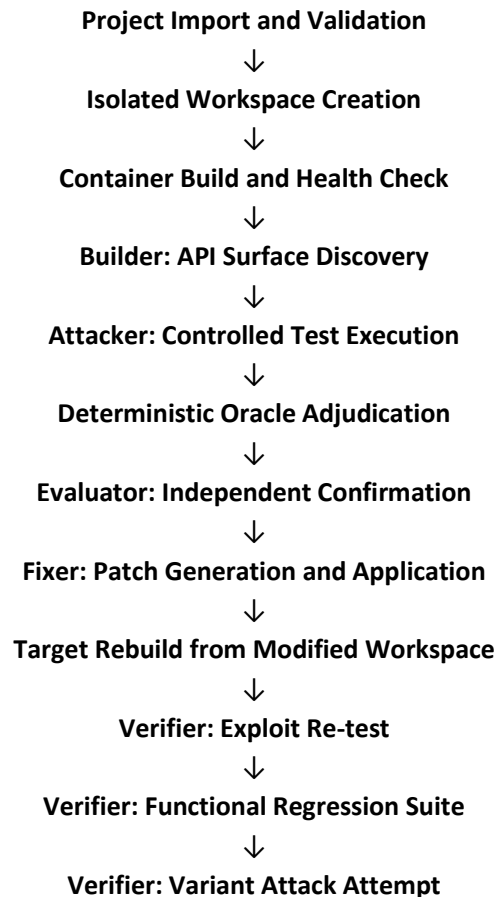
### METHODOLOGY

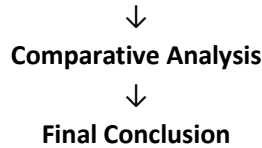
#### 3.1 Research Design

This study follows a quantitative experimental design. A software platform is constructed and then evaluated under controlled conditions against benchmark applications whose vulnerabilities are known in advance.

The experiment begins with a FastAPI application that is imported, validated and copied into an isolated workspace. A container is built from that workspace and started under restrictive runtime settings. The Builder agent then derives the application's route inventory, authentication schemes and ownership relationships. The Attacker agent selects and executes controlled test cases against the running container, capturing the full request and response for each. The Evaluator agent receives only that captured evidence and adjudicates each candidate as confirmed, rejected or inconclusive. The Fixer agent generates a unified diff for each confirmed finding and applies it within a separate modified workspace. The target is then rebuilt from the patched workspace and the Verifier re-runs both the original exploit and the application's functional test suite before recording a final outcome.

The overall research process is:





### 3.1.1 Research Design Components

| Component              | Description                                                                                              |
|------------------------|----------------------------------------------------------------------------------------------------------|
| Independent variables  | Target application under test and the vulnerability class being exercised                                |
| Dependent variables    | Detection outcome, adjudication status, patch validity and verification outcome                          |
| Pipeline agents        | Builder, Attacker, Evaluator, Fixer, and the Verification component                                      |
| Vulnerability classes  | Broken Object Level Authorization, mass assignment, security misconfiguration                            |
| Benchmark conditions   | Vulnerable application versus functionally equivalent secured application                                |
| Adjudication method    | Deterministic platform-code oracles operating on captured request and response pairs                     |
| Severity method        | CVSS v3.1 base score computed from evidence-cited enumerated inputs                                      |
| Verification criterion | Dual criterion: original exploit blocked and functional test suite still passing                         |
| Evaluation metrics     | Detection rate, false discovery rate, fix accuracy, functional regression rate, post-fix robustness rate |
| Main outcome           | Confirmed findings supported by reproducible evidence, with verified remediation status                  |

### 3.1.2 Database Design / Data Structure

Because the platform must retain sufficient structured evidence to reproduce any reported finding, scan data is stored in MongoDB rather than being held only in memory or written as unstructured logs. Records are correlated by a scan identifier, and the collections that carry experimental meaning are treated as append-only so that a corrected record supersedes an earlier one instead of overwriting it.

The principal collections are as follows. The projects collection holds the imported application's metadata, validation outcome and isolation state. The scans collection records one document per pipeline execution, including its state, stage and counters. The test\_cases collection stores every request issued by the Attacker together with the captured response and the oracle result, whether or not a violation was found, since this forms the denominator for the tests-executed measure.

The findings collection holds one document per candidate vulnerability, carrying its category, CWE identifier, normalised route template, severity inputs and computed score, confidence band, adjudication status and the reason recorded for that status. The patches collection stores the generated unified diff for each file along with the pre-application validation results. The verification\_results

collection records the outcome of re-testing, with the original exploit result, the functional suite result and the variant attack result held in separate fields so that they cannot be conflated.

Size limits are enforced at the repository layer rather than by convention. Captured request and response bodies are truncated beyond a defined threshold with a flag recording that truncation occurred, and oversized patches are rejected outright rather than silently shortened, since a partially applied diff could behave differently from the one that was reviewed.

## 3.2 Data Sources

Rather than drawing on a public vulnerability corpus, the primary evaluation data for this study consists of benchmark FastAPI applications constructed specifically for the research. This decision follows directly from the contamination concern identified in Chapter 2: if the target application and its corresponding fixes exist in public repositories, a language model may reproduce a remembered patch rather than derive one, and the resulting measurement would not reflect genuine capability.

The primary benchmark is a deliberately vulnerable notes-and-identity API. It provides document storage and user profile management, uses bearer-token authentication with two distinct test identities, and contains three planted vulnerabilities recorded at the time of injection together with their category, CWE identifier, route template, method and affected source line range.

A second application accompanies it, exposing the same endpoints with the same behaviour but with each vulnerability correctly remediated. This secured variant functions as the false-positive control: any finding reported against it is by construction incorrect. Including it is essential, because a system evaluated only against vulnerable code can achieve a perfect detection rate simply by reporting every endpoint as vulnerable.

A third fixture is intentionally unbuildable, declaring a dependency that does not exist in any package index. It is used to confirm that a project which cannot be containerised fails predictably into a recorded import-failed state with the build output retained, rather than hanging or aborting the pipeline.

## 3.3 Tools and Technologies Used

6. The orchestration backend is implemented in Python using FastAPI, with Pydantic models providing typed validation at every boundary between components.
7. Docker provides target isolation. Each target runs in a container built from the scan workspace, attached to an internal network, with all Linux capabilities dropped, privilege escalation disabled, a non-root user, a read-only root filesystem and explicit memory, CPU and process limits.
8. MongoDB stores the structured scan record. Indexes are declared in code and asserted at startup so that they cannot depend on manual creation on an individual machine.
9. The language model is reached through a provider abstraction that no agent bypasses. Structured output is mandatory, prompts are stored as versioned files with a registry digest, and every call is recorded with its model identifier, token usage and cost.
10. httpx is used for all controlled test traffic, wrapped in a scope-locked client that rejects any request whose host is not the current scan target and refuses cross-host redirects.

11. Flutter and Dart provide the desktop client, using Riverpod for state management and go\_router for navigation, communicating with the backend over an authenticated local HTTP interface.
12. pytest is used for the platform's own test suite and for executing the benchmark application's functional regression suite during verification.
13. Development is carried out in Visual Studio Code with the backend and desktop client maintained in a single repository alongside the specification documents.

### **3.4 Dataset Description**

Three fixture applications support the experimental work, each serving a distinct purpose.

#### **3.4.1 Vulnerable Benchmark Application**

The vulnerable benchmark exposes document and profile endpoints under bearer-token authentication and provisions two separate test identities so that cross-user access can be attempted. Three vulnerabilities are planted within it.

The first is a Broken Object Level Authorization flaw on the document retrieval endpoint. The handler authenticates the caller correctly but omits the comparison between the document's owner identifier and the authenticated user, so any authenticated identity can retrieve any document. The second is a mass assignment flaw on the profile update endpoint, where the request model permits additional fields and the resulting payload is written to stored state, allowing privilege escalation through an undeclared role field. The third is a security misconfiguration comprising an unauthenticated debug endpoint that discloses internal configuration values, debug mode enabled, and a permissive cross-origin policy combined with credentialed responses.

Each planted vulnerability is accompanied by an executable proof of vulnerability that returns a boolean exploited result together with structured evidence. Each proof is verified to succeed against the vulnerable build and to fail against a hand-corrected reference build before the application is admitted to the dataset, ensuring that the oracle itself is sound.

The application additionally ships a functional test suite covering its legitimate behaviour, including a user reading their own document, creating a document, updating their own profile biography, being refused when updating another user's profile, and being refused when unauthenticated. This suite passes against the vulnerable build and provides the second condition of the dual-criterion verification.

#### **3.4.2 Secured Control Application**

The secured control application implements the same endpoints and the same legitimate behaviour, with each of the three vulnerabilities correctly remediated: ownership is verified before a document is returned, the profile request model forbids undeclared fields, and the debug endpoint is removed with debug mode disabled and the cross-origin policy restricted.

This application supplies the negative condition required to measure precision. The same proofs of vulnerability are executed against it and are required to report that no exploitation occurred. Any finding reported against this application is counted as a false positive without further adjudication.

### **3.4.3 Import-Failure Fixture**

The third fixture is a syntactically valid FastAPI application whose dependency manifest requires a package that does not exist. It cannot be built, and exists solely to confirm that the isolation pipeline handles unbuildable projects predictably. The expected behaviour is a recorded import-failed state carrying a truncated but informative extract of the build output, with all partially created resources cleaned up.

### **3.4.4 Dataset Selection for the Experiment**

The vulnerable benchmark serves as the primary experimental target because it is the only fixture for which exact ground truth exists by construction. The secured control is used in every evaluation run alongside it, since detection performance reported without a corresponding false-positive measurement is not interpretable. The import-failure fixture is excluded from detection metrics and is used only to exercise the failure path.

## **3.5 Agent Selection and Responsibilities**

Five components perform the analytical work of the pipeline. Four are language-model agents with distinct responsibilities, and the fifth is a deterministic verification component. The division is intended to keep each stage independently inspectable rather than to increase the number of model invocations.

### **3.5.1 Builder Agent**

The Builder establishes what the target application actually exposes. It first performs static analysis of the source using Python's abstract syntax tree module to extract route decorators, HTTP methods, path parameters, dependency declarations and Pydantic model configurations. It then retrieves the OpenAPI schema from the running container, which is authoritative where available.

The language model is invoked only for what static analysis cannot determine, namely the semantics of authentication dependencies and the identification of which parameters denote resource ownership. Deriving the route table deterministically rather than through the model is a deliberate decision: the normalisation of concrete paths to route templates underpins the later matching of findings to ground truth, and must therefore be reproducible.

### **3.5.2 Attacker Agent**

The Attacker selects applicable test categories for each discovered endpoint and executes controlled requests against the running container. Every executed test is persisted regardless of its outcome, since tests that produced no finding form the denominator for the tests-executed measure and are required for replay.

Crucially, the Attacker does not decide whether a test succeeded. That determination is made by a deterministic oracle implemented in ordinary program code, which inspects the recorded request and response pair. For the authorization category, for example, the oracle requires a 2xx status together with a response body containing the other user's discriminating owner identifier; a 2xx response with an empty or filtered body is not treated as a finding. This separation is the central methodological

commitment of the study, because an oracle implemented in a prompt would measure the model's consistency rather than the system's detection capability.

All traffic passes through a scope-locked client that refuses any request outside the current target, enforces a per-scan rate limit and request ceiling, and blocks destructive HTTP verbs. Rate-limit findings therefore demonstrate the absence of a limit rather than attempting to exhaust the target.

### **3.5.3 Evaluator Agent**

The Evaluator adjudicates each candidate finding independently. It receives the captured request, the captured response and the oracle result, and explicitly does not receive the Attacker's narrative or the target's source code. This restriction is what makes the independence structural rather than nominal, and it also reduces the exposure of the adjudicating stage to instructions embedded in the target source.

The Evaluator re-executes the test to confirm reproducibility; a candidate that cannot be reproduced becomes inconclusive and never confirmed. It also performs an internal consistency check, rejecting evidence that contradicts the asserted category, such as an authorization claim in which the returned owner identifier in fact matches the requesting principal.

Severity is not emitted by the model. The Evaluator supplies CVSS version 3.1 base metric inputs drawn from closed enumerations, each accompanied by a citation to specific evidence, and the platform computes the vector string, numeric score and severity band using the published equations. Storing the inputs alongside the score allows any score to be recomputed if the rubric is later revised. Confidence is likewise derived from structured signals rather than emitted as a model-supplied value.

### **3.5.4 Fixer Agent**

The Fixer maps each confirmed finding to its location in the workspace inventory and produces a unified diff. The model proposes the diff; it never executes a command and never writes to disk. Platform code parses and applies the diff, which is the operative constraint should the model be influenced by injected instructions.

Every model-supplied path is resolved and rejected unless it falls within the modified workspace, symbolic links are not followed, and a target path absent from the Builder's inventory is recorded as a hallucinated path without any write being attempted. Before application, the patched file must parse as valid Python, and diff size and file-count ceilings are enforced. These validation results are stored on the patch record as data so that a rejected patch remains analysable rather than merely logged.

Patches are written exclusively to a modified workspace that is separate from both the user's original project and the unmodified scan workspace, preserving the required progression from original to scan copy to patched copy.

### **3.5.5 Verification Component**

Verification rebuilds the target from the modified workspace and re-runs the original exploit using the same deterministic oracle. It then executes the application's functional test suite and compares the result against the pre-fix baseline.

An outcome of fixed is recorded only when both conditions hold. Where the exploit is blocked but a previously passing functional test now fails, the outcome is recorded as regressed, which is a distinct state and not a form of success. Where the exploit still succeeds the outcome is unresolved, and where the rebuild itself fails the outcome is unverified. After a fixed outcome is established, one adapted variant of the original attack is attempted and its result is stored in a separate field, distinguishing a patch that closes a single exploit path from one that addresses the underlying weakness.

### **3.5.6 Overall Reason for the Selected Division**

The five components correspond to genuinely different tasks rather than to arbitrary subdivision. Understanding an application, attempting to exploit it, judging the resulting evidence, repairing the defect and confirming the repair require different inputs and produce different outputs, and each can fail in a different way.

Separating them yields two properties that a single-agent design cannot provide. The component that judges a finding does not have access to the reasoning that produced it, so confirmation is not self-referential. Equally, the component that repairs code operates on a confirmed finding supported by runtime evidence, rather than on a suspicion, which reduces the incidence of patches generated for defects that were never demonstrated.

## **3.6 Framework / Workflow Diagram / Architecture**

The platform is organised in two layers. A Flutter desktop client provides the user interface, and a Python backend owns all orchestration, agent execution, container management and source modification. The client holds no security logic, no prompts and no database credentials; it presents state supplied by the backend and issues commands to it over an authenticated interface bound to the loopback address.

A scan proceeds through clearly separated stages. On import, the project is statically validated without executing any of its code, since executing an untrusted project merely because it was selected would defeat the purpose of the tool. On isolation, the source is copied into a scan workspace and a container is built and health-checked. Because the target attaches to an internal network with no published ports, health checks and subsequent test traffic are routed through a scanner-side proxy container attached to both the internal network and a bridge, so that the target itself never becomes reachable from the host network.

The agent pipeline then executes in sequence, with each stage persisting its output before the next begins, so that an interrupted scan leaves a partial but coherent record. Because container builds and other blocking operations would otherwise stall the event loop, long-running work is executed as supervised background jobs with cancellation tokens, and the client observes progress by polling job state rather than by inferring it.

Three invariants hold across the whole architecture. The user's original project is never modified, a property asserted by comparing a recursive checksum of the source tree before and after a complete run. No finding is reported as confirmed without a stored request and response demonstrating it. And no model output is permitted to become an executable action: model output is a typed proposal, and platform code decides whether that proposal is permitted before anything is executed, written or requested.



## REFERENCES

- [1] Wang, X. et al., "CVE-Bench: Benchmarking LLM-based Software Engineering Agent's Ability to Repair Real-World CVE Vulnerabilities," Proceedings of NAACL, 2025.
- [2] Zhang, Y. et al., "VulAgent: Hypothesis-Validation based Multi-Agent Vulnerability Detection," arXiv preprint arXiv:2509.11523, 2025.
- [3] Beurer-Kellner, L. et al., "Design Patterns for Securing LLM Agents against Prompt Injections," arXiv preprint arXiv:2506.08837, 2025.
- [4] OWASP Foundation, "OWASP API Security Top 10 - 2023 Edition," 2023. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- [5] Ullah, S. et al., "Do Language Models Reason About Security Vulnerabilities?" Proceedings of the IEEE Symposium on Security and Privacy, 2024.
- [6] OWASP Foundation, "OWASP Zed Attack Proxy (ZAP) Documentation," 2024. [Online]. Available: <https://www.zaproxy.org/docs/>
- [7] FIRST.org, "Common Vulnerability Scoring System version 3.1: Specification Document," 2019. [Online]. Available: <https://www.first.org/cvss/v3.1/specification-document>
- [8] MITRE Corporation, "Common Weakness Enumeration (CWE) List Version 4.14," 2024. [Online]. Available: <https://cwe.mitre.org/>